

Rapid #: -24663591

CROSS REF ID: **19186259890004361**

LENDER: **CPO (California State Polytechnic Univ., Pomona) :: Main Library**

BORROWER: **B3G (Ben-Gurion University of the Negev, Aranne Central Library) :: Main Library**

TYPE: Article CC:CCG

JOURNAL TITLE: Journal of mechanisms and robotics

USER JOURNAL TITLE: Journal of mechanisms and robotics

ARTICLE TITLE: Targeted Sampling Dynamic Window Approach: A Path-Aware Dynamic Window Approach
Sampling Strategy for Omni-Directional Robots

ARTICLE AUTHOR:

VOLUME: 16

ISSUE: 11

MONTH:

YEAR: 2024-11-01

PAGES:

ISSN: 1942-4302

OCLC #:

Processed by RapidX: 5/28/2025 10:40:12 AM

This material may be protected by copyright law (Title 17 U.S. Code)



Targeted Sampling Dynamic Window Approach: A Path-Aware Dynamic Window Approach Sampling Strategy for Omni-Directional Robots

Chen Shen

Engineering Product Development,
Singapore University of Technology and Design,
Singapore 487372
e-mail: chen_shen@mymail.sutd.edu.sg

Gim Song Soh¹

Engineering Product Development,
Singapore University of Technology and Design,
Singapore 487372
e-mail: soh.gimsong@sutd.edu.sg

This paper describes a sampling strategy for the dynamic window approach (DWA), a local path planner, for omni-directional robot motion planning. An efficient local planner allows the robot to quickly respond to dynamic obstacles and ensures that commanded velocities meet the dynamic constraints of the robot. While typical DWA implementations sample the velocity space evenly, we propose that targeted sampling (TS) will result in a more fine-grained search of the relevant velocity space, leading to better control and performance in space-constrained environments. Our TS-DWA strategy is informed by the global planned path, allowing us to sample more velocities in the general path direction. We employ a polar velocity generator to selectively sample velocities and couple angular velocity samples to the path curvature. A bias for angular velocity is added for robots with a preferred heading, such as robots with forward-mounted sensors, to quickly turn toward the desired direction for better sensing. The strategy is implemented as a robot operating system (ROS) navigation stack local_planner plugin and tested in simulation with Gazebo using an omni-directional robot platform. Experiments show that as the space around the simulated robot gets smaller, our proposed sampling strategy results in more successful navigation trials to the goal in space-constrained environments compared to other commonly used methods like DWA and timed-elastic-band, where planning fails or oscillates. TS-DWA was also deployed on the Vision60 quadrupedal robot to demonstrate navigation through narrow corridors in the real world. [DOI: 10.1115/1.4065073]

Keywords: legged robots, mobile robots, robot motion planning and control

1 Introduction

Path planning and following is an important part of any robot's navigation stack. They allow the robot to determine the path the robot should travel to reach its goal, and the corresponding control commands that should be sent to the robot locomotion system to execute the planned path. Due to the computational complexity of the navigation problem, a hierarchical structure is typically used, with the global planner determining the general path to the desired goal, and the local planner determining the velocity commands required to reach a sub-goal along the planned path, accounting for robot dynamic constraints.

Sampling-based methods such as the dynamic window approach (DWA) [1] sample in the velocity space. To account for the

dynamics of the robot, the velocity space is constrained based on the reachable velocities of the robot given maximum acceleration and deceleration within a pre-defined time-step, which generates admissible control commands. However, the quality of the solution depends on the number of samples taken.

Existing DWA implementations tend to sample evenly in the velocity space, which results in performant solutions for non-holonomic robots, where the search space is greatly reduced due to holonomic constraints. However, with the increasing viability of omni-directional robots, such as legged quadrupedal robots, sampling-based solutions will need to sample in the extra dimension that the robot is able to move in. Given the same computation power, the same number of samples can be checked for viability. This however results in lower sample resolution due to the extra possible dimension for sampling, which leads to worse solutions, especially in space-constrained environments.

In this paper, we propose the use of the global planned path to better direct our search in the velocity space, allowing for more targeted sampling (TS) of velocities that move the robot along the global planned path, while using sparse samples in other directions

¹Corresponding author.

Contributed by the Mechanisms and Robotics Committee of ASME for publication in the JOURNAL OF MECHANISMS AND ROBOTICS. Manuscript received October 29, 2023; final manuscript received March 1, 2024; published online March 29, 2024. Assoc. Editor: Dongming Gan.

for overcoming invalid global plans due to previously unseen or dynamic obstacles. This results in more fine-grained sampling in the likely direction of travel, without too high of a computational cost, compared to simply increasing the number of samples across all axes.

2 Related Work

There are few approaches for solving the local planning problem. The first are sampling-based methods. DWA [1] is one such sampling method, where the robot dynamics are used to determine the range of possible robot translational and angular velocities that are possible within a predetermined simulation period to constrain the search space. Samples are then taken within the range of admissible velocities and rolled out into trajectories. Resulting trajectories that lead to collision are dropped, and the non-collision trajectories are scored against an objective function to determine the best trajectory. The objective function uses the global plan generated path, and an occupancy grid map [2] or cost-map [3] to evaluate each velocity sample. The corresponding velocities used to generate the best scoring trajectory are then applied to the robot. Modifications to DWA have been proposed, such as in Refs. [4,5], which use a deep reinforcement learning-based approach to modify DWA parameters during run-time based on the environmental condition. Missura and Bennewitz [6] implement a dynamic obstacle detector that allows for DWA to predict collision with moving obstacles. Howard et al. [7] propose sampling in the state space of the robot instead, in contrast to DWA, with the global planned path informing the local search of the state space for complex environments.

Elastic band approaches have also been proposed, such as in Refs. [8,9], where the desired trajectory is modeled as an elastic band, bending around obstacles and contracting within itself to reduce the path length. Reference [10] is a proposed modification of the elastic band approach, which is an optimization-based method to determine a trajectory and time between points along the trajectory that account for the dynamic constraints of the robot.

Potential field approaches have also been proposed, such as in Ref. [11], where obstacles result in a repulsive force on the robot, with the magnitude of the force being inversely proportional to the squared distance between the robot and obstacle. The goal point applies an attractive force onto the robot. The robot thus navigates the environment, avoiding obstacles, by following the virtual force applied onto the robot. Naively using the potential field for navigation would result in the robot getting trapped in local minima, however, and several modifications have been proposed to get around these issues. Sarvesh et al. [12] propose the use of the global planned path to modify the obstacle potential field used for local planning, avoiding a few potential trap situations.

Finally, learning-based methods have also been explored for navigation in cluttered environments, such as in Ref. [13], which employs deep neural networks for sensor data processing and future prediction, and are trained in a supervised fashion based on collected data. Another deep neural network is used to learn a policy function for choosing velocity commands through simulated trials using reinforcement learning. Xiao et al. [14] present a self-supervised learning approach that learns to map environment configurations to their respective motion plans. Many learning methods, however, lack explainability [15], which make improvements difficult if an undesirable plan is produced.

DWA, with its velocity space search, ensures that robot dynamics are accounted for naturally when searching for a trajectory. However, with the increased viability of omni-directional robot platforms such as quadruped robots, the additional control space requires more samples and thus more computational power to maintain the same sample density in the search space. By using the global plan to inform sampling, increased sampling density and thus finer control can be realized in regions of interest, and decreased in others, without greatly impacting computation load.

3 Targeted Sampling Approach

Our proposed solution builds on the sampling-based approach described in Ref. [1], where a change is made in the sampling strategy, by only sampling velocities that move the robot along the global path plan, rather than the entire reachable velocity window. Similar to Ref. [16], we employ a polar representation for the translational velocity search space, with samples characterized by their heading and magnitude. To extract the desired heading direction, we look at a sub-goal along the global plan, constraining our search space to reduce computational cost. Since the global plan does not update as frequently as the local planner requires, our local planner samples a range of translational velocities that are within the general direction of the desired heading, with a magnitude between the minimum and maximum allowable translational magnitude based on robot performance parameters. The resulting space of trajectories searched is represented by the shaded gray region in Fig. 1, while the dotted line denotes the trajectory arc from the global plan information extraction, which is used to center the search. We also include a few other samples for collision avoidance, such as moving left, right, and reverse relative to the path direction.

The path curvature is extracted by examining the first few points of the global plan. A bias is added to the curvature for omni-directional robots with a preferred heading direction, such as those with forward-facing sensors. This has the effect of turning the robot toward the direction of travel, reducing the possibility of reversing along the path. The angular velocity sample is then determined by the estimated path curvature.

To ensure that the global plan information extraction does not result in a large computational penalty, the approximation methods below were chosen for determining the path heading and path curvature.

3.1 Global Plan Information Extraction. To inform our sampling strategy using the global plan, we first extract important features, such as the path heading to a look-ahead waypoint and the path curvature required to reach that waypoint and beyond from the global plan, as shown in Figs. 2 and 3, respectively.

To see how this is done, we first assume that the global plan is available as an array of N waypoints, with $\mathbf{X}^{\text{GP}} = \{(X_0, Y_0)^T, (X_1, Y_1)^T, \dots, (X_{\text{goal}}, Y_{\text{goal}})^T\}$ defined in the coordinate frame of the map. To determine the required heading direction, we look along the global plan $\mathbf{X}^{\text{GP}}$, and choose a look-ahead waypoint $\mathbf{X}_{i_{\text{look}}}^{\text{GP}}$, with i_{look} denoted as the look-ahead index, as shown in Fig. 2. The directional vector $\mathbf{a}$ from the current robot position $\mathbf{X}_{\text{pos}}$ to the

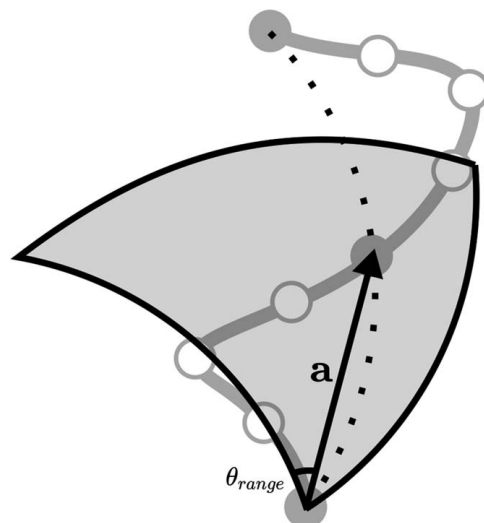


Fig. 1 Reduced velocity search space informed by global plan

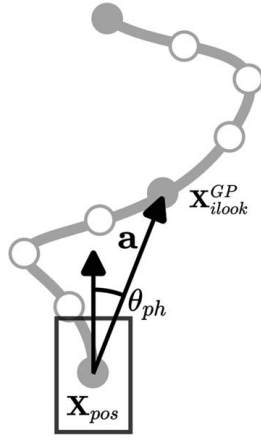


Fig. 2 Path heading approximation using look-ahead index along global plan

look-ahead waypoint $\mathbf{X}_{look}^{GP}$ is found and transformed into the local coordinate frame of the robot:

$$\mathbf{a} = [R(\phi_{map})]^{-1}(\mathbf{X}_{look}^{GP} - \mathbf{X}_{pos}) \quad (1)$$

where $[R(\phi_{map})]$ defines the current planar pose of the robot in the frame of the map. The direction vector $\mathbf{a}$ is then used to determine the require heading θ_{ph} in the robot frame using

$$\theta_{ph} = \arctan \frac{a_y}{a_x} \quad (2)$$

The implementation of this extraction is detailed in Algorithm 1.

Algorithm 1 Path heading extraction

```

 $N \leftarrow \text{length of } \mathbf{X}^{GP}$ 
 $j \leftarrow i_{look}$ 
 $\phi_{map} \leftarrow \text{robot yaw in map frame}$ 
If  $j \geq N$  then
     $j \leftarrow N - 1$  ▷ Last point used for look-ahead
end if
 $\mathbf{X}_j \leftarrow \mathbf{X}_j^{GP}$ 
 $\mathbf{X} = \mathbf{X}_j - \mathbf{X}_{pos}$ 
 $R^{-1} \leftarrow \begin{bmatrix} \cos(\phi_{map}) & \sin(\phi_{map}) \\ -\sin(\phi_{map}) & \cos(\phi_{map}) \end{bmatrix}$  ▷ Create rotation matrix
 $\mathbf{a} \leftarrow R^{-1}\mathbf{X}$  ▷ Path vector in robot frame
 $\theta_{ph} \leftarrow \arctan \left( \frac{a_y}{a_x} \right)$  ▷ Path heading in robot frame
return  $\theta_{ph}$ 

```

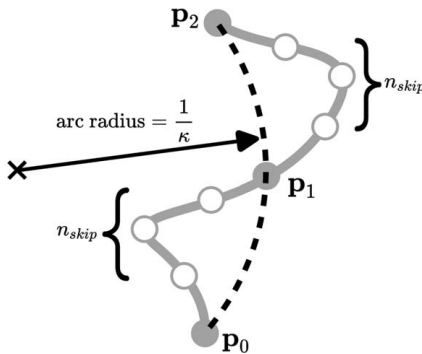


Fig. 3 Path curvature approximation, skipping points between each point chosen in global plan

Next, to generate a desired trajectory to the look-ahead point, three points $\mathbf{p}_0$, $\mathbf{p}_1$, and $\mathbf{p}_2$ along the global plan are taken to calculate the required path curvature as shown in Fig. 3, starting from the robot current position $\mathbf{p}_0$ to the look-ahead point $\mathbf{p}_1$, and subsequently $\mathbf{p}_2$, a waypoint on the global plan that was skipped with the same number of points (n_{skip}) as the look-ahead point.

A circular arc is then fit through these points based on the Menger curvature κ to extract the path curvature information:

$$\kappa = \frac{4A}{|\mathbf{p}_0 - \mathbf{p}_1||\mathbf{p}_2 - \mathbf{p}_3||\mathbf{p}_3 - \mathbf{p}_1|} \quad (3)$$

where A denotes the area of the triangle spanned by the three points. This can be computed easily using Eq. (4).

$$A = \frac{1}{2}(\mathbf{p}_2 - \mathbf{p}_0) \times (\mathbf{p}_1 - \mathbf{p}_0) \quad (4)$$

The implementation of which is detailed in Algorithm 2.

Algorithm 2 Path curvature extraction

```

 $\kappa \leftarrow 0$ 
 $N \leftarrow \text{length of } \mathbf{X}^{GP}$ 
 $i \leftarrow 0$ 
 $j \leftarrow 1 + n_{skip}$  ▷ Index of second point
 $k \leftarrow 2 + 2n_{skip}$  ▷ Index of third point
If  $jk \leq N$  then
     $\mathbf{p}_0 \leftarrow \mathbf{x}_i^{GP}$ 
     $\mathbf{p}_1 \leftarrow \mathbf{x}_j^{GP} - \mathbf{p}_0$  ▷ Make  $\mathbf{p}_0$  be origin
     $\mathbf{p}_2 \leftarrow \mathbf{x}_k^{GP} - \mathbf{p}_0$ 
     $A \leftarrow \frac{(\mathbf{p}_{1,x})(\mathbf{p}_{2,y}) - (\mathbf{p}_{1,y})(\mathbf{p}_{2,x})}{2}$  ▷ Area of triangle
     $\kappa \leftarrow \frac{4A}{|\mathbf{p}_1||\mathbf{p}_1 - \mathbf{p}_2||\mathbf{p}_2|}$ 
end if
return  $\kappa$ 

```

3.2 Polar Sample Generator. Now, with the heading and path curvature information to the look-ahead point along the global plan in the previous step, we generate a set of targeted velocity samples, which consists a set of angular velocity ω and translational velocity $\mathbf{v}_{trans}$, for trajectory generation and scoring by the local path planner.

To generate the set of translational velocity samples, we populate n_{asamp} evenly spaced heading samples from the range $\theta_{ph} \pm \theta_{range}$, and n_{vsamp} evenly spaced velocity magnitude samples based on the robot minimum and maximum translational velocity, $v_{trans, min}$ and $v_{trans, max}$, respectively. To convert these samples into robot motion commands, we decomposed the generated samples, which is in the polar form, into their x-axis and y-axis components:

$$\mathbf{v}_{trans} = v_{trans} \begin{bmatrix} \cos \theta_{ph} \\ \sin \theta_{ph} \end{bmatrix} \quad (5)$$

Next, to generate the angular velocity samples, we utilize the geometric relationship that the translational velocity vector has with its angular velocity, as the robot travels at a constant speed along a circular trajectory. The angular velocity ω is related to the translational velocity v_{trans} and path curvature κ by

$$\omega = v_{trans}\kappa \quad (6)$$

To encourage the robot to turn toward the path heading, a curvature bias proportional to the offset between the robot heading and path heading, θ_{ph} , is added.

$$\omega = v_{trans}(\kappa + \alpha_{ph}\theta_{ph}) \quad (7)$$

The strength of this bias is controlled by a non-negative parameter, α_{ph} . To see why this turns the robot heading to path direction, we first consider the movement of a robot translating in a circle, with zero angular velocity (Fig. 4(a)). In this scenario, the robot

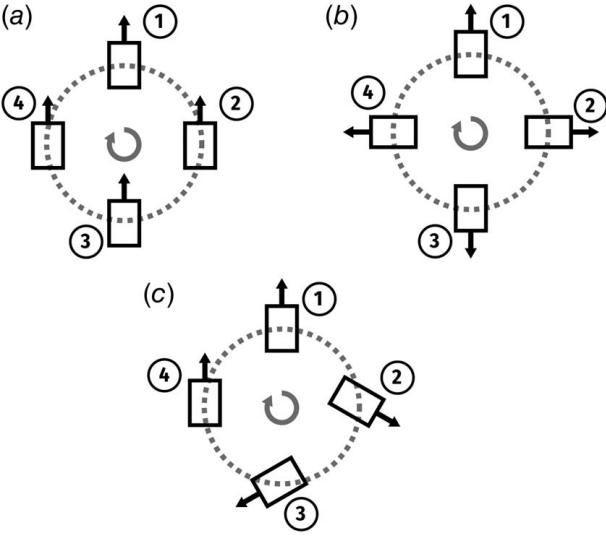


Fig. 4 Effects of the addition of bias to curvature: (a) translation tangential to path, (b) addition of angular velocity $\propto$ curvature, and (c) additional bias to curvature

heading is unchanged. Next, we introduce an angular velocity to the robot, using Eq. (6). The resulting motion leaves the relative heading between the circle and the robot constant (Fig. 4(b)). Finally, introducing the bias to the angular velocity, Eq. (7), causes the robot heading to converge to the tangent of the circle (Fig. 4(c)), with the magnitude of α_{ph} affecting how much additional correction to the heading is done.

Lastly, a check is done to ensure the generated v_x and v_y samples fall within the robot respective $v_{trans, min}$ and $v_{trans, max}$ limits. If the limits are exceeded, they are set to the closest limit. The same check and limit correction is also done for ω . The resultant triple of the velocity sample $\mathbf{v} = (v_x, v_y, \omega)$ is added to the sample buffer. Three other samples are also added for the case where $\theta_{ph} = \pm 90$ deg and $\theta_{ph} = 180$ deg, so that the robot will have some options to move left, right, or reverse relative to the desired path direction. To account for the exits of narrow passages, where it might be desirable for the robot to turn only when it has properly cleared the narrow passage, an additional sample of the calculated translational velocity in Eq. (5) is appended with zero angular velocity, such that $\mathbf{v} = (v_x, v_y, 0)$ is added to the sample buffer. This process is detailed in Algorithm 3.

Algorithm 3 Sample generation

```

 $\Theta \leftarrow \{n_{asamp} \text{ evenly spaced heading samples}\}$ 
 $M \leftarrow \{n_{vsamp} \text{ evenly spaced magnitude samples}\}$ 
Sample buffer  $\leftarrow \{\}$ 
While  $\Theta$  not Empty do
   $\theta \leftarrow \text{pop element from } \Theta$ 
   $M_{temp} \leftarrow M$ 
  While  $M_{temp}$  not Empty do
     $m \leftarrow \text{pop element from } M_{temp}$ 
     $v_x \leftarrow \text{constrain}(m \cos \theta, v_{x, min}, v_{x, max})$ 
     $v_y \leftarrow \text{constrain}(m \sin \theta, v_{y, min}, v_{y, max})$ 
    Sample buffer append  $(v_x, v_y, 0)$ 
     $\omega \leftarrow \text{constrain}(m(\kappa + \alpha_{ph}\theta_{ph}), \omega_{min}, \omega_{max})$ 
    Sample buffer append  $(v_x, v_y, \omega)$ 
  end while
end while
return Sample buffer

```

3.3 Sample Evaluation. Similar to Ref. [1], each velocity sample in the sample buffer is rolled out into a trajectory,

$\mathbf{X}^{LP}$, which is an array of T waypoints, $\{(X_0, Y_0)^T, (X_1, Y_1)^T, \dots, (X_T, Y_T)^T\}$. The trajectory roll-out assumes that the velocity is held constant. The resultant trajectories are checked against the footprint of the robot and the local cost-map of the environment, M^{occ} , which encodes the position of obstacles, such that trajectories that result in collision are discarded. The remaining non-collision trajectories are scored against an objective function. The same objective function used in the robot operating system (ROS) navigation stack implementation of DWA is employed in our proposed local planner, where the trajectory is scored against three different criteria: (1) the distance from the end of the trajectory to the global planned path, (2) the distance from the end of the trajectory to the last waypoint, $(\mathbf{X}_k^{GP})$, in the global plan still within the local cost-map, and (3) the maximum obstacle cost along the trajectory. These criteria are combined with different weights to form the final objective function, $G(*)$. The velocity triple that generated the best scoring trajectory is then used to command the robot.

$$G(*) = \beta PD(\mathbf{X}^{LP}, \mathbf{X}^{GP}) + \gamma GD(\mathbf{X}^{LP}, \mathbf{X}^{GP}) + \delta OD(\mathbf{X}^{LP}, M^{occ})$$

$$\text{where } PD(\mathbf{X}^{LP}, \mathbf{X}^{GP}) = \min_i \|\mathbf{X}_T^{LP} - \mathbf{X}_i^{GP}\|, i \in \{0 \dots N\} \quad (8)$$

$$GD(\mathbf{X}^{LP}, \mathbf{X}^{GP}) = \|\mathbf{X}_T^{LP} - \mathbf{X}_k^{GP}\|$$

$$OD(\mathbf{X}^{LP}, M^{occ}) = \max_{(X_t, Y_t)} M_{(X_t, Y_t)}^{occ}, t \in \{0 \dots T\}$$

The overall pseudocode for the proposed planner is detailed in Algorithm 4.

Algorithm 4 TS-DWA local planner

```

 $\theta_{ph} \leftarrow \text{heading\_extract}(\mathbf{X}^{GP})$  ▷ Algorithm 1
 $\kappa \leftarrow \text{curvature\_extract}(\mathbf{X}^{GP})$  ▷ Algorithm 2
Sample buffer  $\leftarrow \text{buffer\_gen}(\theta_{ph}, \kappa)$  ▷ Algorithm 3
cmd_vel  $\leftarrow \text{pop element from sample buffer}$ 
cmd_cost  $\leftarrow G(\text{cmd\_vel})$  ▷ Eq. (8)
While Sample buffer not Empty do
  sample  $\leftarrow \text{pop element from sample buffer}$ 
  cost  $\leftarrow G(\text{sample})$  ▷ Eq. (8)
  If cost < cmd_cost then
    cmd_cost  $\leftarrow$  cost
    cmd_vel  $\leftarrow$  sample
  end if
end while
return cmd_vel

```

4 Experiments and Results

In this section, we seek to evaluate the performance of our proposed path-aware DWA planner with the existing DWA and timed-elastic-band (TEB) local planner as the robot moves through a variety of environment with narrow segments, with as little as 10% width gap for the robot to maneuver through.

4.1 Simulation Setup. Experiments were carried out in the Maze, Graph, and Track environments from Ref. [17] as shown in Fig. 5. These environments were chosen due to the availability of narrow segments along the evaluated routes among others for our trials. Simulations were carried out in Gazebo 11 and ROS Noetic on a computer equipped with an Intel i9-12900H CPU and a RTX3070Ti Mobile GPU. The robot model used for simulation is a modified version of an omni-directional robot platform

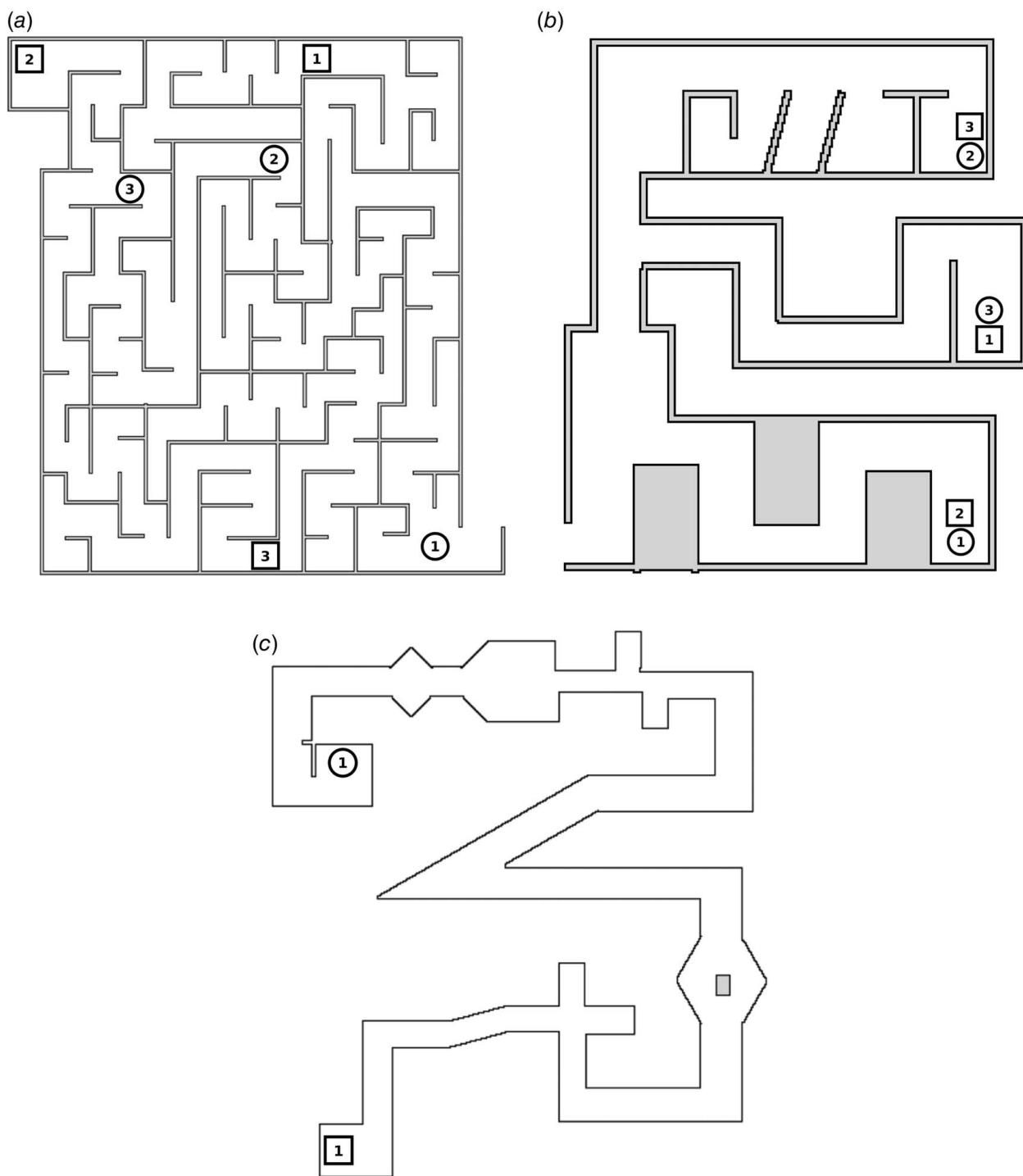


Fig. 5 Test environments and routes (circle: start position, square: desired goal, number: route number): (a) Maze environment, (b) Graph environment, and (c) Track environment

in Ref. [18]. A 360 deg laser scanner was mounted to the top of this robot platform to provide obstacle sensing. Similar to Ref. [17], the ROS navigation stack [19] was used to ensure the algorithms are compared on equal footing. The global planner is kept constant across all trials to ensure fairness during comparison. All local planner parameters were also kept constant across the different environments and routes. For testing in the Maze and Graph environment, three routes were chosen as shown in Figs. 5(a) and 5(b). Similarly for the Track environment, one route was chosen as shown in Fig. 5(c). These routes were selected as they cover the range of features available in each environment.

To test the effectiveness of our proposed approach in environments with narrow segments, the footprint of the robot was increased incrementally such that its width was 50%, 80%, and 90% of the narrowest segment along the route. The length of the robot was also adjusted accordingly to maintain the same aspect ratio of the simulated robot platform in use. For each choice of the chosen robot width along each route, five runs were carried out using the TEB, DWA, and our proposed local planner, TS-DWA, and evaluated against the metrics of motion efficiency, computational efficiency, and velocity smoothness as introduced in Ref. [17]. The motion efficiency measures the time taken for

the robot to reach the desired goal position from its starting position. The computational efficiency measures the average time taken for the local planner to generate a velocity command, and the velocity smoothness measures the average acceleration experienced by the robot.

For the test environment, a 5.5 m by 5.5 m cost-map is used for the local planner, with an increased resolution of 0.02 m compared to 0.1 m used in Ref. [17], as the increased cost-map resolution is required when navigating through narrow passages. The global planner and its parameters are kept the same as in Ref. [17]. The parameters in Table 1 were used for the local planner, with all other local planner parameters set at their defaults. The parameter

α_{ph} was chosen experimentally based on the response of the robot. A larger magnitude would result in a faster response to changes in the desired path heading. Too large a value, however, would saturate the maximum angular acceleration, leading to undesirable oscillation of the robot heading.

4.2 Results

4.2.1 Maze Environment. As shown in Tables 2–4, when the robot width was of a smaller proportion of the corridor width (50%) in the Maze environment, TEB tends to perform the best, with both the fastest time to goal (Table 2) and smoothest velocity

Table 1 Experimental parameters

Parameter (robot)	$v_{trans, max}/v_{trans, min}$	$v_{x, max}/v_{x, min}$	$v_{y, max}/v_{y, min}$	$\omega_{max}/\omega_{min}$	$a_{x, max}/a_{x, min}$	$a_{y, max}/a_{y, min}$	$\alpha_{max}/\alpha_{min}$
Value	0.55/0.1	0.55/−0.2	0.3/−0.3	0.5/−0.5	2.5/−2.5	1.0/−1.0	3.2/−3.2
Units	$m s^{-1}$	$m s^{-1}$	$m s^{-1}$	$rad s^{-1}$	$m s^{-2}$	$m s^{-2}$	$rad s^{-2}$
Parameter (TS-DWA)	θ_{range}	n_{asamp}	n_{vsamp}	α_{ph}	i_{look}	n_{skip}	γ
Value	$\frac{\pi}{4}$	15	10	4.0	4	3	20
Units	rad	—	—	—	—	—	—
Parameter (DWA)	γ	Parameter (TEB)	$min_obstacle_dist$				
Value	20	Value	0.05				
Units	—	Units	m				

Table 2 Motion efficiency (s) and number of successful runs

	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA
	Maze Route 1			Graph Route 1			Track Route 1		
50%	99.36 (5)	111.35 (5)	129.61 (5)	114.92 (5)	75.75 (5)	91.12 (5)	285.70 (5)	150.53 (5)	164.80 (5)
80%	— (0)	139.46 (5)	165.22 (5)	— (0)	88.26 (5)	108.46 (5)	128.71 (1)	164.02 (5)	187.33 (5)
90%	— (0)	252.83 (5)	217.04 (5)	— (0)	— (0)	146.77 (5)	— (0)	— (0)	— (0)
	Maze Route 2			Graph Route 2					
50%	96.55 (5)	115.12 (5)	121.28 (5)	60.68 (5)	65.76 (5)	77.00 (5)			
80%	— (0)	157.40 (2)	148.62 (5)	— (0)	80.54 (5)	105.37 (5)			
90%	— (0)	— (0)	— (0)	— (0)	— (0)	269.54 (2)			
	Maze Route 3			Graph Route 3					
50%	164.27 (5)	121.80 (5)	129.57 (5)	74.69 (5)	62.01 (5)	111.83 (5)			
80%	— (0)	143.80 (5)	165.17 (5)	— (0)	72.29 (1)	85.76 (5)			
90%	— (0)	177.74 (5)	207.70 (5)	— (0)	— (0)	— (0)			

Note: Best performance are in bold

Table 3 Computational efficiency (10^{-3} s)

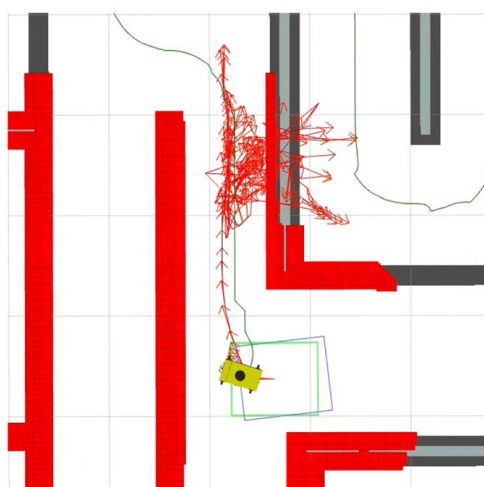
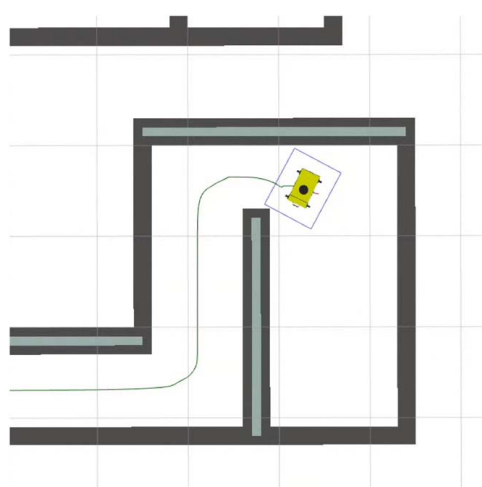
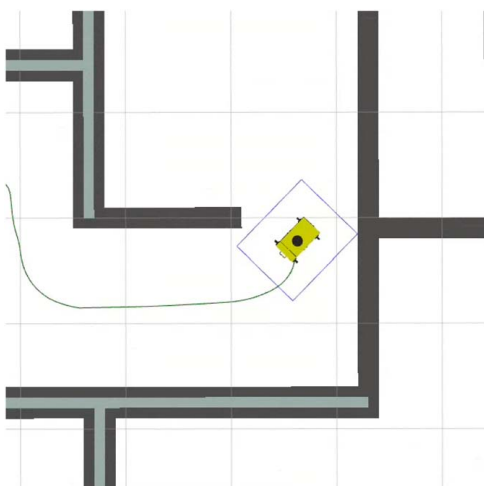
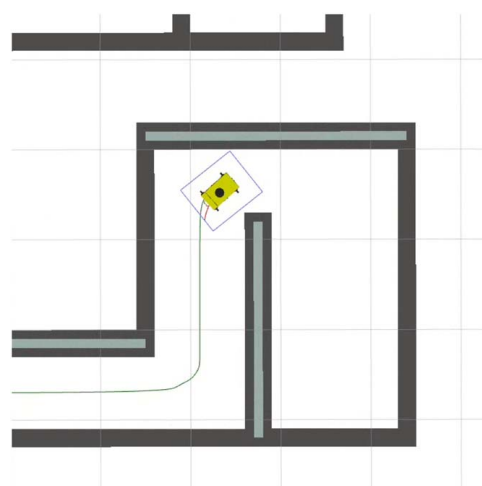
	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA
	Maze Route 1			Graph Route 1			Track Route 1		
50%	19.49	11.14	13.17	48.01	9.93	26.73	62.52	18.30	20.44
80%	—	12.30	12.64	—	10.14	25.35	117.37	20.08	18.61
90%	—	8.51	14.49	—	—	21.15	—	—	—
	Maze Route 2			Graph Route 2					
50%	19.78	10.82	13.28	29.37	14.50	17.56			
80%	—	8.74	12.70	—	15.98	15.02			
90%	—	—	—	—	—	10.99			
	Maze Route 3			Graph Route 3					
50%	51.17	9.69	12.43	65.16	13.16	17.33			
80%	—	10.79	11.85	—	12.80	19.74			
90%	—	9.60	10.79	—	—	—			

Note: Best performance are in bold

Table 4 Velocity smoothness (ms^{-2})

	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA	TEB	DWA	TS-DWA
	Maze Route 1			Graph Route 1			Track Route 1		
50%	0.0512	0.101	0.148	0.0835	0.126	0.152	0.0401	0.0562	0.126
80%	—	0.0881	0.130	—	0.0974	0.129	0.083	0.0645	0.138
90%	—	0.114	0.106	—	—	0.0941	—	—	—
	Maze Route 2			Graph Route 2					
50%	0.0472	0.0758	0.146	0.0434	0.0827	0.141			
80%	—	0.0629	0.134	—	0.0773	0.128			
90%	—	—	—	—	—	0.100			
	Maze Route 3			Graph Route 3					
50%	0.0787	0.091	0.148	0.0901	0.112	0.150			
80%	—	0.0949	0.131	—	0.0763	0.122			
90%	—	0.0714	0.103	—	—	—			

Note: Best performance are in bold

**Fig. 6 TEB invalid through-wall plan****Fig. 8 DWA unstable plan at narrow entrance****Fig. 7 DWA early rotation at narrow passages****Fig. 9 TS-DWA successful entry**

commands (Table 4). This comes at the cost of reduced computational efficiency (Table 3). However, as the robot gets wider (80%), TEB starts to fail in planning, generating impossible plans that extend through the walls of the maze as shown in Fig. 6. On

the other hand, the sampling-based methods continue to generate valid velocity commands, with DWA showing better motion efficiency than our proposed strategy since it searches across the entire velocity space evenly. However, for certain areas of the

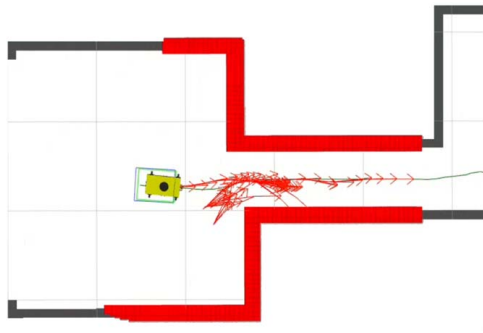


Fig. 10 TEB recoverable planning failure

maze, DWA will occasionally rotate while attempting to navigate narrower passages, eventually resulting in planning failure. This is as shown in Fig. 7, and can be seen from the results for the Maze Route 2 at 80% robot width in Table 2, where DWA was only successful completing its task 2 out of 5 times. With an even wider robot (90%), our proposed strategy starts to outperform DWA, reaching the navigation goal even in instances where DWA fails. This implies that the targeted sampling of velocity space based on the global plan leads to an increased ability to find finer control commands that can help a robot to navigate through these narrow spaces.

4.2.2 Graph Environment. As shown in Tables 2–4, the performance of the various local planners in a Graph environment show a similar general trend. It seems that our proposed local planner completes the navigation task across more runs, as the robot width increases, compared to TEB and DWA. However, the computational efficiency is lower than that of DWA, but higher than TEB, which is expected. In areas where there is a larger space right before a narrow entrance, DWA tends to move toward the entrance, but does not find a valid sample through the entrance, leading to unstable planning between the contrasting objectives of moving along the global plan and moving away from obstacles as shown in Fig. 8. Nevertheless, our proposed approach is able to find a valid sample once near the narrow entrance, leading to a successful entry as shown in Fig. 9.

4.2.3 Track Environment. As shown in Tables 2–4, the performance of the various local planners in the Track environment were handled successfully by all the local planners tested when the robot was at 50% the width of the corridor. Notice that the TEB had recoverable planning failures occur during navigation, as shown in Fig. 10, which leads to a decreased motion efficiency as compared to the other two methods. As the robot became wider (80%), both the DWA and our proposed approach were able to successfully reach the desired goal at a better motion efficiency in general. It is also interesting to note that the computational efficiency of our proposed method in this

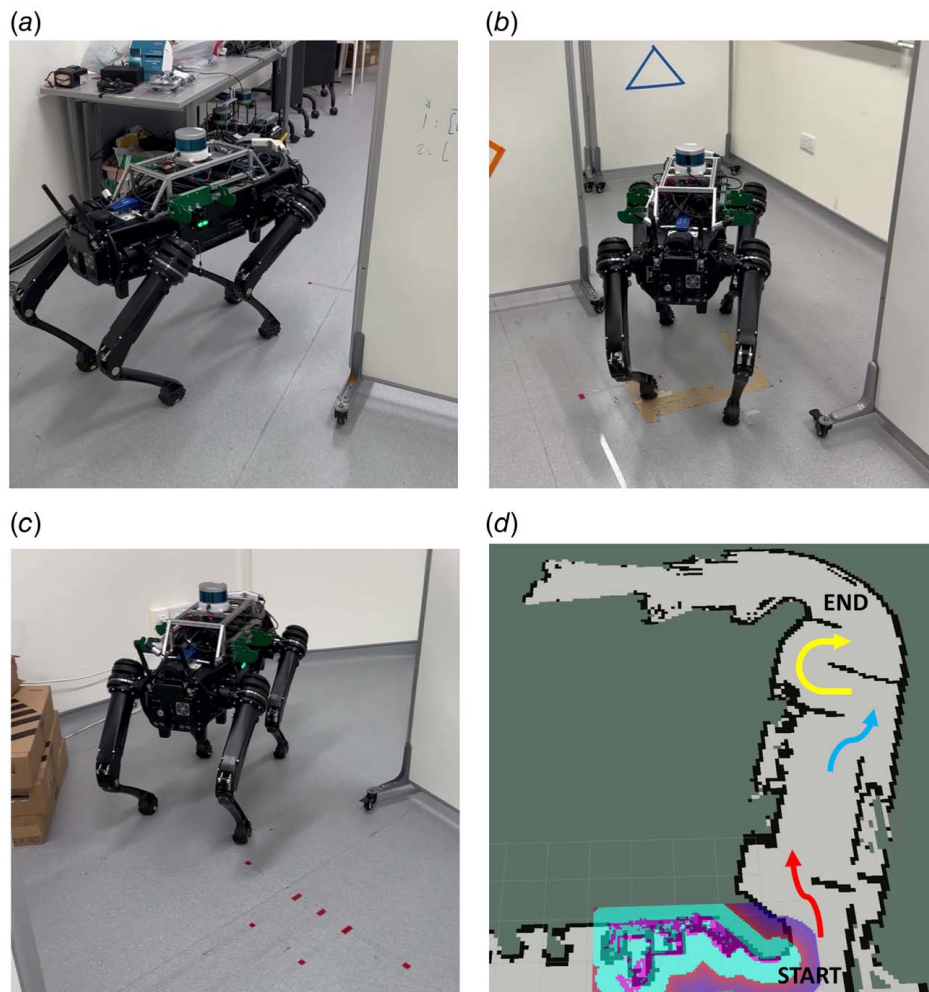


Fig. 11 TS-DWA tested on the Ghost Vision60 robot. (a) First turn (bottom arrow near start), (b) second turn (middle arrow), (c) hairpin turn (top arrow near end), and (d) mapped environment with start and end locations.

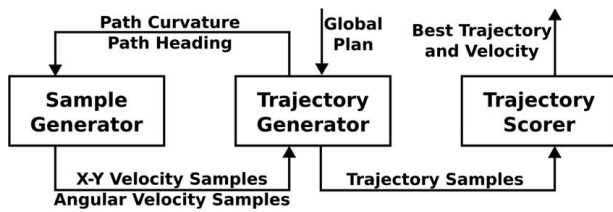


Fig. 12 Flow diagram of the TS-DWA implementation

environment is slightly better than DWA, while TEB had the highest computation time due to many repeated recoverable planning failures.

4.3 Discussion. When the environment is wider relative to the robot width at 50%, TEB has the best performance in velocity smoothness, which mirrors the findings in Ref. [17]. As the robot becomes wider, TEB starts to fail in planning, creating invalid plans that go through obstacles. While the recovery behavior for TEB is sometimes able to find a valid plan eventually after a few resets (which results in a much reduced motion efficiency), the robot usually ends up oscillating in position, while executing the impossible plan. The planning time also increases significantly, as seen in some of the computational efficiency data, which can negatively impact the performance of other software running on the robot.

At 80% of the environment width, TEB is no longer able to successfully complete the navigation task. DWA in this situation is able to perform better, since valid samples could still be found during navigation.

At 90% of the environment width, our proposed approach is still able to navigate some environments where DWA fails, due to our more targeted choice of samples, allowing for more fine control of the robot.

Computational efficiency of our proposed method is slower than DWA due to the overhead of extracting global plan information, but is still more efficient than TEB. The overall bottlenecks in the computation between DWA and TS-DWA were the evaluation of samples, since the cost function is evaluated for each sample, while global information extraction is only done once per iteration.

However, due to the reduced search space, the proposed approach tends to perform worse in motion efficiency than both DWA and TEB when the width of the robot is smaller relative to the width of the environment. The approach also suffers from a reduced velocity smoothness compared to the other two methods.

4.4 Implementation on a Physical Robot. To further test if the proposed local planner functions well beyond simulation, trials were done on a quadrupedal robot (Ghost Robotics Vision60). A passageway was setup using a group of vertical panels and was arranged such that the passage width differs throughout.

An occupancy grid map was first generated based on 2D laser scan using ROS simultaneous localization and mapping (SLAM) package (`slam_gmapping`) by teleoperating the robot through the passageway setup (Fig. 11(d)). The 2D laser scans were generated from a Velodyne VLP-16 Light Detection and Ranging (LIDAR) sensor pointcloud with the `pointcloud_to_laserscan` ROS package. Robot odometry was provided by the A-LOAM LIDAR odometry ROS package.

The robot was then moved to a starting location at the passageway entrance and the ROS Navigation Stack was used for waypoint navigation toward the specified goal at the passageway exit. This was performed with the proposed TS-DWA local planner generating velocity commands based on the global plan. The velocity commands are then sent to the robot's low level gait planner to

command each legs' joints such that the robot's body moves at the desired velocity.

The proposed TS-DWA is implemented as a `move_base` local planner plugin in the ROS Navigation Stack. The plugin is made up of a few components: the sample generator, trajectory generator, and trajectory scorer. The sample generator uses the polar representation of the search space to generate samples. It converts them into x - y axis velocities as required by the trajectory generator module. The trajectory generator calculates the path curvature and heading as required by the sample generator module. The resulting samples from the generator are then used to generate trajectories for scoring. The trajectory scorer uses the original DWA cost function, which takes into account the trajectory's endpoint to the global plan waypoint, the maximum obstacle cost along the trajectory, and the distance from the trajectory to the global planned path, as detailed in Eq. (8). A diagram for the implemented TS-DWA local planner plugin is depicted in Fig. 12.

From the trial, it was found that the robot was able to navigate from the start to the goal without any collisions or unrecoverable planning failures, even with some localization inaccuracies introduced by the SLAM and the odometry algorithms. This provides a view of the proposed local planner's performance for navigating through narrow passages when applied onto a physical robot running the ROS navigation stack and its typical dependencies. Figure 11 shows the image sequence of the robot navigating through the passage setup, passing through the various tight corners as seen in Figs. 11(a)–11(c).

5 Conclusion

Across all three simulation environments, our approach is better able to successfully navigate to the goal when the robot width is large, at up to 90% of the narrowest width of the corridor along the path of the robot, where DWA and TEB fails. It also reduces the number of unexpected in-place rotations (not caused by recovery behavior) and oscillations compared to DWA. We also show TS-DWA moving through a narrow corridor setup in the real world on a physical robot. Future work involving the use of alternate methods for sampling around the desired path heading and curvature could result in striking a better balance between exploring the full velocity search space and exploiting global plan information. This would help to improve both motion efficiency and velocity smoothness. Further exploration into switching between informed sampling and uninformed sampling based on the environment may also yield better performance. Other potential areas for improvement include the addition of other terms into the cost function for the sampling process, which could lead to selections that result in better motion efficiency and velocity smoothness. Accounting for energy efficiency as a performance metric would lead to further comparison between the proposed planner and existing work. Lastly, while the local planner assumes that the robot is able to achieve the commanded velocities in a relatively short time, further work on a system-level analysis that includes the modeling of the robot's inertial parameters and form of locomotion, such as using omni-directional wheels or legs, might lead to deeper insights into the overall performance of each planner when implemented on different robots.

Conflict of Interest

There are no conflicts of interest.

Data Availability Statement

The datasets generated and supporting the findings of this article are obtainable from the corresponding author upon reasonable request.

References

- [1] Fox, D., Burgard, W., and Thrun, S., 1997, "The Dynamic Window Approach to Collision Avoidance," *IEEE Rob. Autom. Mag.*, **4**(1), pp. 23–33.
- [2] Elfes, A., 1989, "Using Occupancy Grids for Mobile Robot Perception and Navigation," *Computer*, **22**(6), pp. 46–57.
- [3] Lu, D. V., Hersherberger, D., and Smart, W. D., 2014, "Layered Costmaps for Context-Sensitive Navigation," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Chicago, IL, Sept. 14–18, pp. 709–715.
- [4] Dobrevski, M., and Skočaj, D., 2020, "Adaptive Dynamic Window Approach for Local Navigation," *IEEE/RSJ International Conference on Intelligent Robots and Systems*, Las Vegas, NV, Oct. 25–29, pp. 6930–6936.
- [5] Xiao, X., Wang, Z., Xu, Z., Liu, B., Warnell, G., Dhamankar, G., Nair, A., and Stone, P., 2022, "APPL: Adaptive Planner Parameter Learning," *Rob. Auton. Syst.*, **154**, p. 104132.
- [6] Missura, M., and Bennewitz, M., 2019, "Predictive Collision Avoidance for the Dynamic Window Approach," *IEEE International Conference on Robotics and Automation*, Montreal, QC, Canada, May 20–24, pp. 8620–8626.
- [7] Howard, T. M., Green, C. J., Kelly, A., and Ferguson, D., 2008, "State Space Sampling of Feasible Motions for High-Performance Mobile Robot Navigation in Complex Environments," *J. Field Rob.*, **25**(6–7), pp. 325–345.
- [8] Quinlan, S., and Khatib, O., 1993, "Elastic Bands: Connecting Path Planning and Control," *Proceedings IEEE International Conference on Robotics and Automation*, Atlanta, GA, May 2–6, Vol. 2, pp. 802–807.
- [9] Quinlan, S., 1995, "Real-Time Modification of Collision-Free Paths," PhD Thesis, Stanford, CA, UMI Order No. GAX95-16902.
- [10] Rösmann, C., Feiten, W., Wösch, T., Hoffmann, F., and Bertram, T., 2013, "Efficient Trajectory Optimization Using a Sparse Model," *European Conference on Mobile Robots*, Barcelona, Catalonia, Spain, Sept. 25–27, pp. 138–143.
- [11] Borenstein, J., and Koren, Y., 1989, "Real-Time Obstacle Avoidance for Fast Mobile Robots," *IEEE Trans. Syst. Man Cybernetics*, **19**(5), pp. 1179–1187.
- [12] Sarvesh, A., Carroll, A., and Gopalswamy, S., 2022, "Reshaping Local Path Planner," *IEEE Rob. Autom. Lett.*, **7**(3), pp. 6534–6541.
- [13] Hoeller, D., Wellhausen, L., Farshidian, F., and Hutter, M., 2021, "Learning a State Representation and Navigation in Cluttered and Dynamic Environments," *IEEE Rob. Autom. Lett.*, **6**(3), pp. 5081–5088.
- [14] Xiao, X., Liu, B., Warnell, G., and Stone, P., 2021, "Toward Agile Maneuvers in Highly Constrained Spaces: Learning From Hallucination," *IEEE Rob. Autom. Lett.*, **6**(2), pp. 1503–1510.
- [15] Xiao, X., Liu, B., Warnell, G., and Stone, P., 2022, "Motion Planning and Control for Mobile Robot Navigation Using Machine Learning: A Survey," *Auton. Rob.*, **46**(5), pp. 569–597.
- [16] Brock, O., and Khatib, O., 1999, "High-Speed Navigation Using the Global Dynamic Window Approach," *Proceedings IEEE International Conference on Robotics and Automation*, Atlanta, GA, May 2–6, Vol. 1, pp. 341–346.
- [17] Wen, J., Zhang, X., Bi, Q., Pan, Z., Feng, Y., Yuan, J., and Fang, Y., 2021, "Mrpb 1.0: A Unified Benchmark for the Evaluation of Mobile Robot Local Planning Approaches," *IEEE International Conference on Robotics and Automation*, Xi'an, China, May 30–June 5, pp. 8238–8244.
- [18] Baran, R., 2020, "Nexus 4wd Mecanum Simulator," https://github.com/RBinsonB/nexus_4wd_mecanum_simulator.
- [19] Marder-Eppstein, E., Berger, E., Foote, T., Gerkey, B., and Konolige, K., 2010, "The Office Marathon: Robust Navigation in an Indoor Office Environment," *IEEE International Conference on Robotics and Automation*, Anchorage, AK, May 3–8, pp. 300–307.